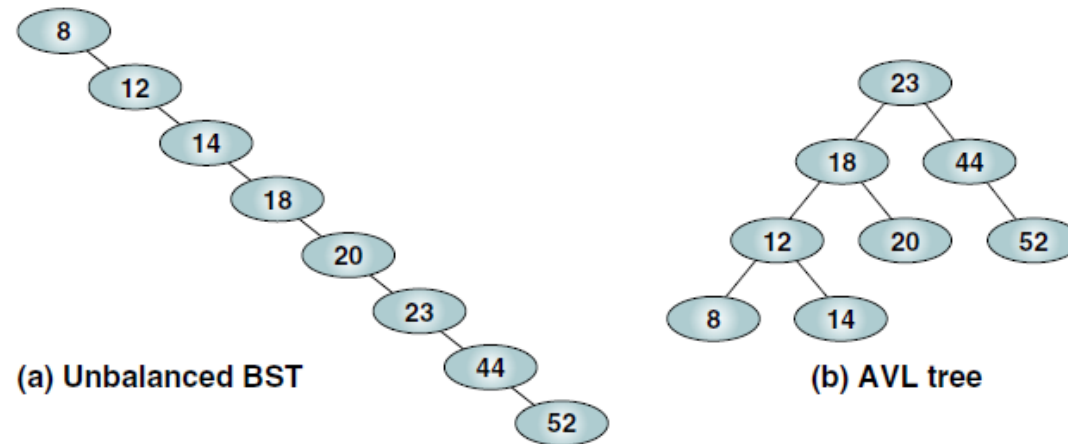


AVL

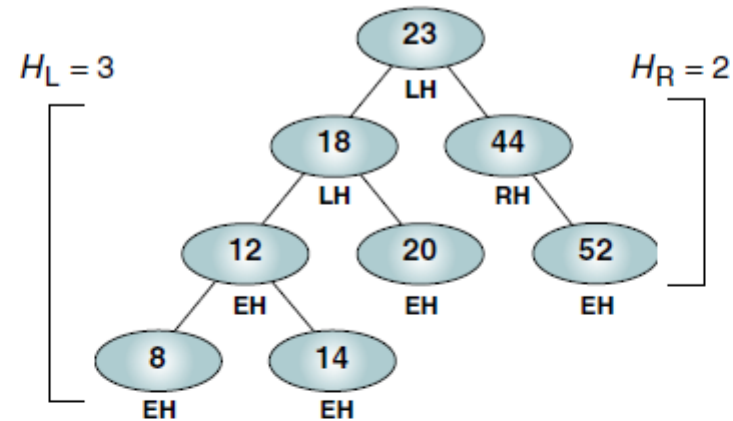


Concept

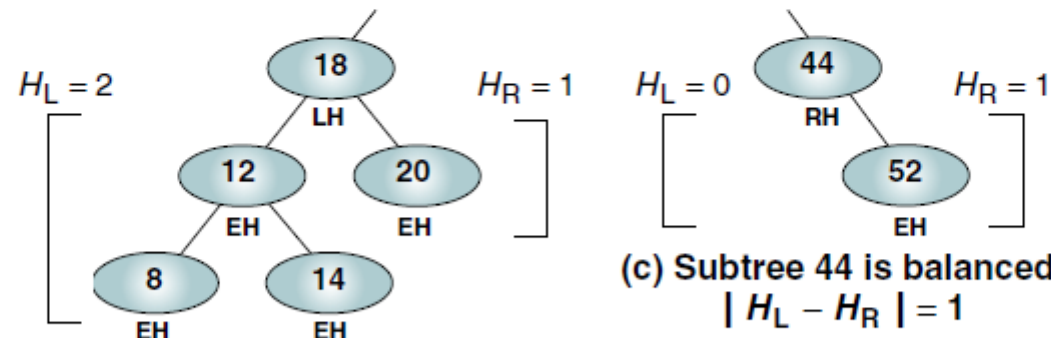
- While the **binary search tree** is simple and easy to understand, it has one major problem: it is **not balanced**.
- An **AVL tree** is a search tree in which the heights of the subtrees differ by no more than 1. It is thus a **balanced binary tree**.



An **AVL tree** is a binary tree that either is empty or consists of two AVL subtrees, T_L and T_R , whose heights differ by no more than 1. $|H_L - H_R| \leq 1$



(a) Tree 23 appears balanced: $H_L - H_R = 1$



(b) Subtree 18 appears balanced:
 $H_L - H_R = 1$

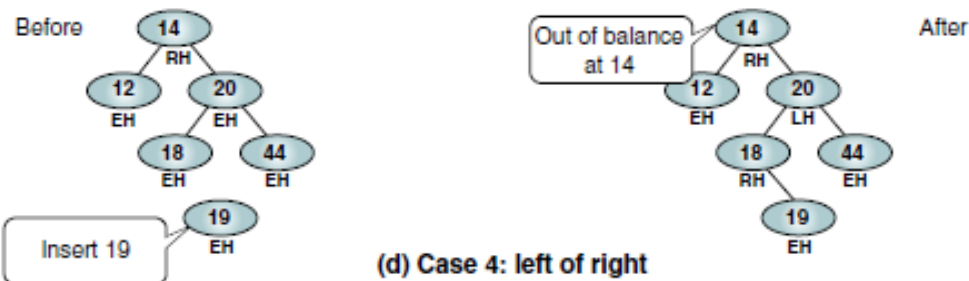
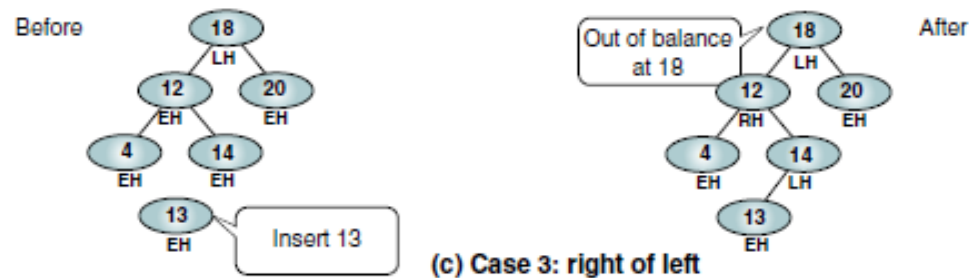
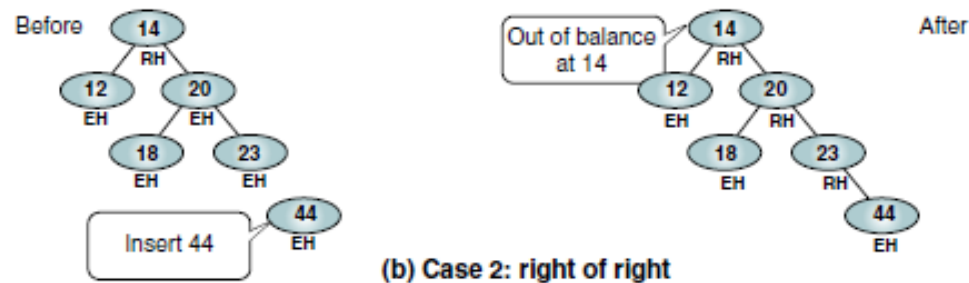
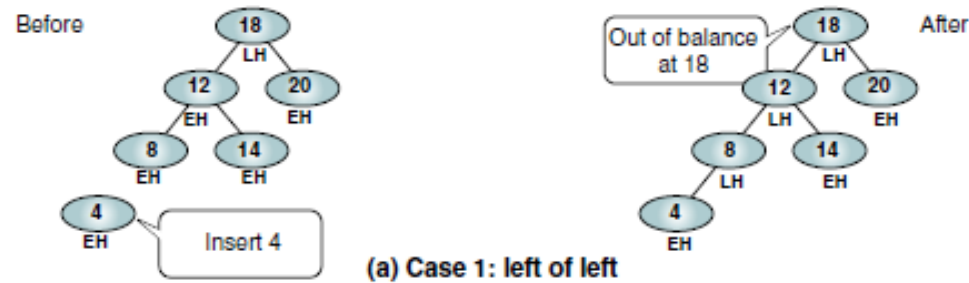
(c) Subtree 44 is balanced:
 $|H_L - H_R| = 1$

Balancing Trees

- Whenever we **insert** a node into a tree or **delete** a node from a tree, the resulting tree may be **unbalanced**.
- If the balance factor ($|H_L - H_R|$) of a node is positive, the node is considered left high. If the balance factor is negative, it's right high.

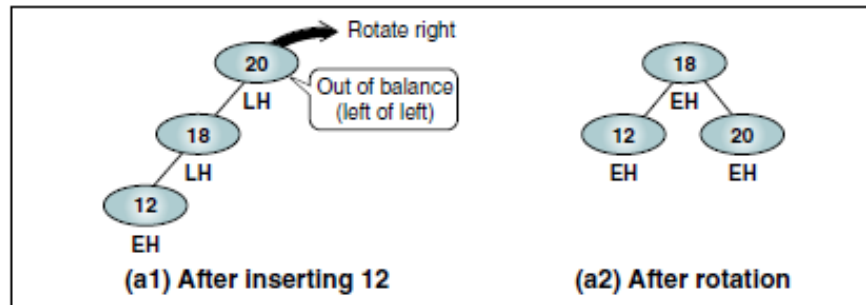
We consider four cases that require rebalancing. All unbalanced trees fall into one of these four cases:

1. **Left of left**—A subtree of a tree that is left high has also become left high.
2. **Right of right**—A subtree of a tree that is right high has also become right high.
3. **Right of left**—A subtree of a tree that is left high has become right high.
4. **Left of right**—A subtree of a tree that is right high has become left high.

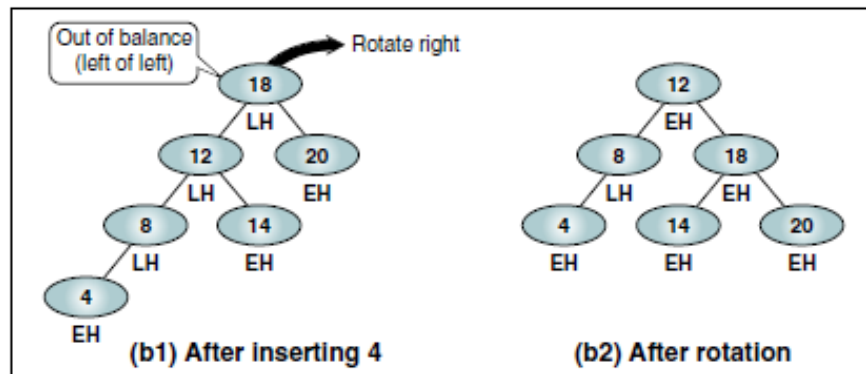


Balancing Trees

- AVL trees are balanced by **rotating nodes** either to the left or to the right. After insertion, the first node (going up) where the balance factor is ± 2 is the **imbalance point**.

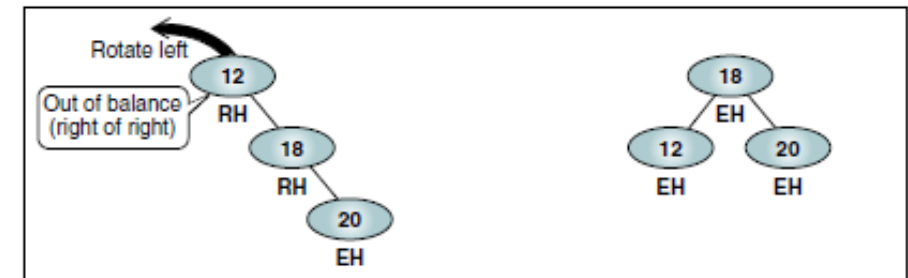


(a) Simple right rotation

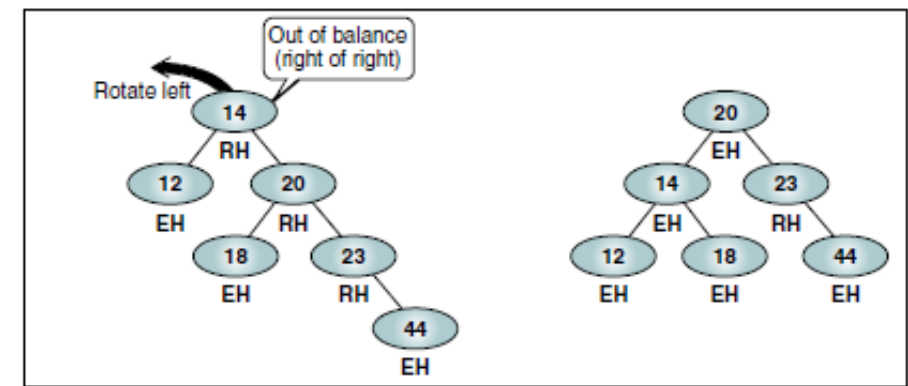


(b) Complex right rotation

Left of Left—Single Rotation Right



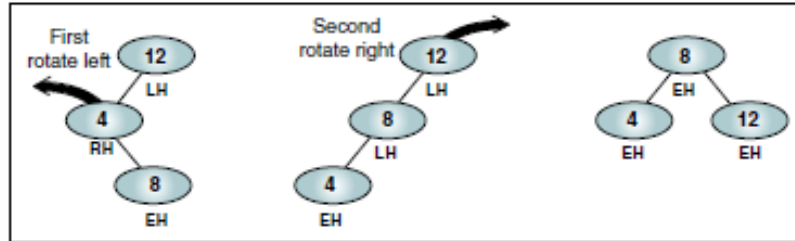
(a) Simple left rotation



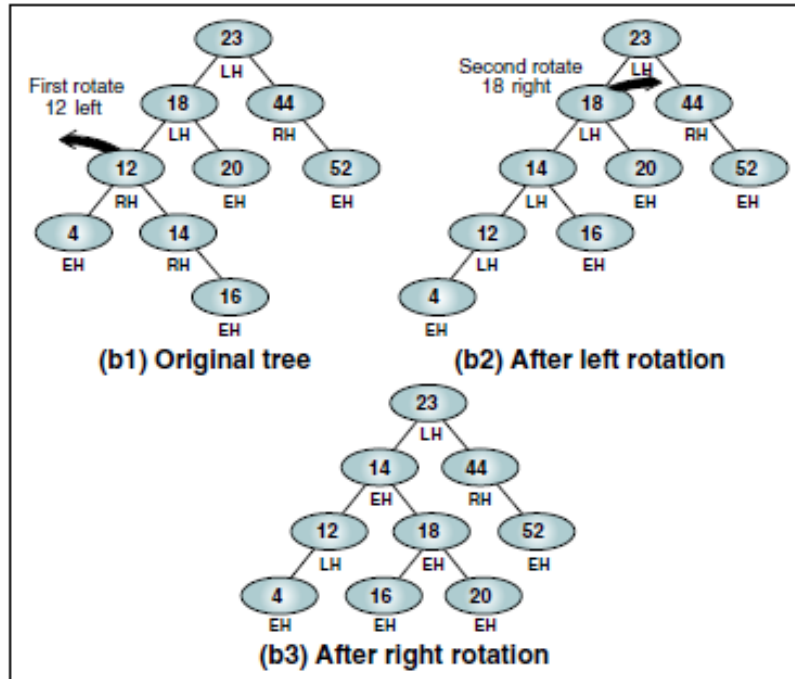
(b) Complex left rotation

Right of Right—Single Rotation Left

Balancing Trees



(a) Simple double rotation right

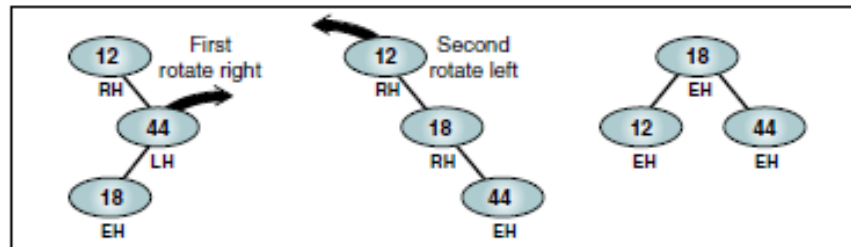


(b) Complex double rotation right

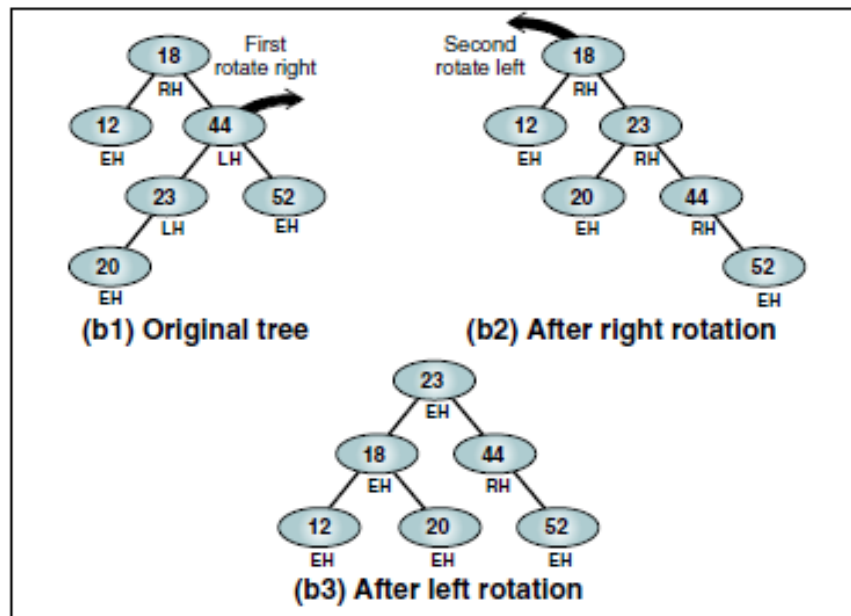
First perform a **left rotation** on the left child, and then perform a **right rotation** on the root.

This is called a **double rotation**.

Balancing Trees



(a) Simple double rotation right



(b) Complex double rotation right

First perform a right rotation on the right child, and then **perform a left rotation** on the root.

This is also a **double rotation**.

AVL Tree Insert

Algorithm AVLInsert (root, newData)

Using recursion, insert a node into an AVL tree.

Pre root is pointer to first node in AVL tree/subtree

newData is pointer to new node to be inserted

Post new node has been inserted

Return root returned recursively up the tree

```
1 if (subtree empty)
  Insert at root
  1 insert newData at root
  2 return root
2 end if
3 if (newData < root)
  1 AVLInsert (left subtree, newData)
  2 if (left subtree taller)
    1 leftBalance (root)
  3 end if
4 else
  New data >= root data
  1 AVLInsert (right subtree, newPtr)
  2 if (right subtree taller)
    1 rightBalance (root)
  3 end if
5 end if
6 return root
end AVLInsert
```

AVL Tree Left Balance

Algorithm leftBalance (root)

This algorithm is entered when the root is left high (the left subtree is higher than the right subtree).

Pre root is a pointer to the root of the [sub]tree

Post root has been updated (if necessary)

```
1 if (left subtree high)
  1 rotateRight (root)
2 else
  1 rotateLeft (left subtree)
  2 rotateRight (root)
3 end if
end leftBalance
```

Rotate AVL Tree Right and Left

Algorithm rotateRight (root)

This algorithm exchanges pointers to rotate the tree right.

Pre root points to tree to be rotated

Post node rotated and root updated

```
1 exchange left subtree with right subtree of left subtree
2 make left subtree new root
end rotateRight
```

Algorithm rotateLeft (root)

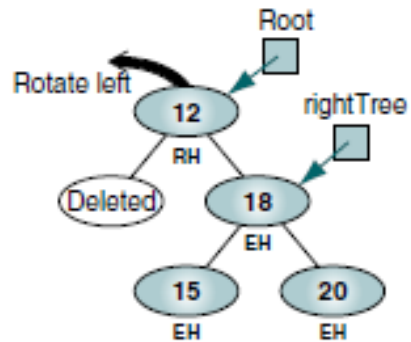
This algorithm exchanges pointers to rotate the tree left.

Pre root points to tree to be rotated

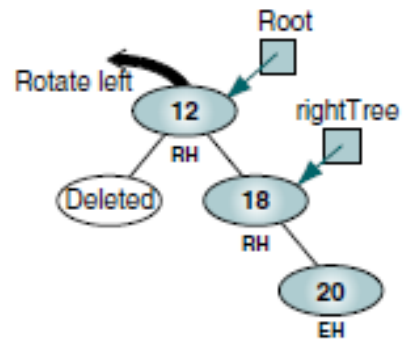
Post node rotated and root updated

```
1 exchange right subtree with left subtree of right subtree
2 make right subtree new root
end rotateLeft
```

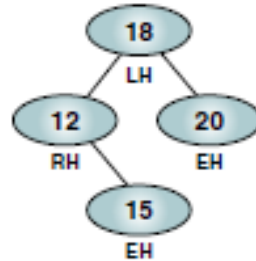
Balancing Trees



(a) Right subtree is even balanced



(b) Right subtree is not even balanced



Deletion in AVL Tree:

1. **Delete the node** as you would in a regular BST.
2. **Update the heights** of the nodes.
3. **Check for imbalances.** If any node becomes unbalanced, **apply the appropriate rotation to restore balance.**

Exercise

1. Create an AVL tree using the following data entered as a sequential set.
Show the balance factors in the resulting tree:

80, 70, 66, 56, 33, 23, 14, 10, 7

2. Insert 44 and 50 into the tree created in Q(1).
3. Delete the node containing 80 from the AVL tree in the following Figure:

